Wind power forecasting under corrupted SCADA for edge deployment: sparse MoE with feasibility guidance

Junyu Li[1], Juntao Du[1,]

[a]*School of Statistics and Applied Mathematics, Anhui University of Finance and Economics, Bengbu, 233030, China*

Abstract

Forecast errors in wind integration alter reserve procurement, curtailment, fallback scheduling, and balancing cost rather than only benchmark scores. In controller-side deployment, short-term wind forecasting must absorb corrupted SCADA telemetry, meet batch-1 edge limits on latency and memory, and keep outputs inside a deployable feasibility envelope. Ultra-LSNT addresses that setting with sparse mixture-of-experts routing and Jump Gate skipping, while FG-MoE adds a training-time envelope regularizer anchored to a last-observation wind proxy that remains available at inference. On Wind (CN), Ultra-LSNT retains positive skill at the severe Gaussian slice ($\sigma_{\text{eff}} = 0.60$) with $R^2 = 0.7863$, while FG-MoE reduces rated-power exceedance from 0.0550% to 0.0092% against a matched no-guardrail backbone without clean/severe accuracy collapse.

Keywords: Wind power forecasting, Economic dispatch, SCADA imperfections, Edge computing, Robustness, Wind power integration

1. Introduction

Wind-power forecasts matter because they are consumed inside operating decisions before they are ever compared in benchmark tables. Studies on renewable offering, storage-coupled scheduling, and OPF under uncertainty show that forecast error is transmitted into reserve procurement, curtailment, balancing actions, and operating cost rather than remaining a purely statistical quantity [**? ? ? ? ?**]. In those settings, larger forecast errors do not merely widen confidence bands; they change reserve volumes, shift dispatch toward more expensive backup or thermal support, and alter curtailment or storage decisions across the operating day [**? ? ?**].

*Corresponding author.

For controller-side forecasting, the relevant question is what the forecast does to the downstream rule, not only how well it compresses to RMSE or R^2.

That operational framing brings the input stream into focus. At wind-farm level, SCADA records are routinely affected by missing intervals, duplicated records, calibration shifts, change points, and alarm ambiguity; recent work on anomaly detection, change-point handling, and power-curve preprocessing treats those defects as first-order obstacles because they alter what can be inferred from the telemetry in the first place [**? ? ? ?**]. In those studies, unflagged regime shifts and corrupted operating points are not benign noise: they move samples across operating regions, distort local normal-behavior baselines, and contaminate the signal used to reconstruct turbine response [**? ? ?**]. A forecasting model validated on sanitized windows can therefore misread corruption as a genuine ramp or inherit the wrong operating regime once the same defects reappear online.

If those corrupted intervals must be handled where they are measured, deployment constraints stop looking like an implementation footnote. Edge-intelligence studies in smart grids and offshore wind repeatedly describe controller-side inference as batch-1, latency-limited, memory-limited, and communication-constrained [**? ?**]. That coupling matters because the intervals most likely to need extra processing are also the ones arriving at devices least able to sustain a dense always-on path. Corrupted inputs plus batch-1 controller deployment therefore make active-path size a design variable rather than a deployment afterthought.

Progress on wind forecasting has been real, but it is distributed across lines of work that are usually validated under different premises. Decomposition and hybrid pipelines help stabilize non-stationary wind series by separating trends, ramps, and oscillatory components, yet those gains are typically reported after the signal has already been regularized by the decomposition or reconstruction pipeline itself [**? ? ? ?**]. Attention-based and graph-enhanced deep models improve representation quality on forecasting benchmarks, but they are often evaluated on cleaned inputs or with no explicit controller-side batch-1 budget in view [**? ? ?**]. Physics-guided studies add turbine envelopes, structural priors, and uncertainty-aware constraints, but several formulations rely on auxiliary states or physical information richer than what deployment can actually observe [**? ? ?**]. A model that must read corrupted SCADA directly, keep a small active path at controller-side batch-1 inference, and obey feasibility signals available at deployment is still rarely the object being tested.

The resulting research question is whether short-term wind forecasting can remain reliable under corrupted controller-side SCADA while staying light enough for batch-1 deployment and disciplined enough to avoid deployment-inconsistent outputs [**? ? ?**]. Ultra-LSNT tackles that setting with sparse routing that reserves more computation for harder windows, while FG-MoE constrains training with feasibility signals that remain available at deployment. The experiments then ask whether this controller-side design stays robust, physically disciplined, and edge-feasible on Wind (CN).

2. Related work

The papers that matter most here do not form a single model inventory [**? ? ?**]. One group asks what a forecaster sees when operational SCADA is corrupted and controller resources are tight; another asks what the forecast must satisfy once it enters a downstream decision layer.

2.1. Operational SCADA Forecasting Under Corruption and Resource Constraints

Most SCADA papers in wind energy are not forecasting papers at all. They are condition-monitoring, anomaly-detection, change-point, and maintenance studies, and that is precisely why they matter here. Reviews and recent surveys show that site-level telemetry typically requires filtering, segmentation, and channel validation before it can support fault diagnosis or performance assessment [**? ? ?**]. Recent anomaly-detection and fleet-level preprocessing studies continue the same pattern: the model is built after the data stream has been stabilized, not before [**? ? ?**]. These papers establish that the telemetry itself is unstable, but they mostly stabilize it for diagnostics, maintenance, or preprocessing rather than ask a forecaster to operate directly on corrupted live input.

A different literature tackles wind variability itself. Decomposition and hybrid forecasting methods report gains by separating scales or regimes in non-stationary wind signals, and those gains are usually measured after the signal has already been regularized by the decomposition or reconstruction pipeline [**? ? ? ? ?**]. Newer deep models shift the emphasis toward learned temporal and graph representations, but they are commonly evaluated on cleaned benchmarks or without an explicit controller-side compute budget [**? ? ?**]. Those studies answer how well models track wind variability after preprocessing or under relatively loose inference budgets, not how they behave when corrupted telemetry reaches batch-1 controller devices.

SCADA studies mostly stabilize telemetry before diagnosis or preprocessing, whereas forecasting papers usually begin after that step. Edge-deployment studies add another condition: controller and field-gateway inference must respect tight latency, memory, and communication budgets [**? ?**]. Evidence is much thinner for forecasters that consume corrupted live SCADA under those same budgets than for diagnostics built after stabilization or forecasting on cleaned signals.

2.2. Feasibility-Aware Forecasting and Decision-Coupled Value

Feasibility enters the literature through a different route. Classical wind-turbine envelopes and power-curve studies use cut-in, rated, and cut-out regions mainly as data-quality filters: implausible samples are identified, removed, or reassigned before the learning problem is posed [**? ? ? ?**]. More recent forecasting work separates more clearly between posthoc repairs that trim outputs after inference and training-time formulations that push physical constraints or uncertainty-aware priors into the objective itself [**? ? ? ?**]. That distinction matters because posthoc repair and training-time discipline do not make the same claim about what the model has actually learned.

Operational-value studies introduce another separation. Reserve scheduling, coordinated offering, and OPF papers repeatedly show that the same forecast quality can translate into different curtailment, congestion, and cost outcomes once the forecast is mapped into a particular scheduling or network model [**? ? ? ? ? ?**]. In that literature, value is mediated by the forecast-to-grid interface and the dispatch rule, not recovered directly from an accuracy ranking.

Physical-discipline papers often assume access to states, envelopes, or auxiliary signals richer than what deployment can actually observe, whereas dispatch-value papers usually start only after the forecast interface and scheduling rule have been fixed [**? ? ? ?**]. The combination that remains rare is training feasibility constraints from deployment-available signals and then comparing the resulting forecasts through one common downstream interface.

3. Methodology

This section defines the model inputs, sparse routing mechanism, feasibility regularizer, and evaluation criteria used in the shared protocol.

3.1. Problem setup

Given a multivariate look-back window $\mathbf{X} \in \mathbb{R}^{L \times C}$ with L historical steps and C SCADA covariates, the task is to predict an H-step active-power trajectory $\hat{\mathbf{Y}} \in \mathbb{R}^{H \times 1}$. The constrained learning problem is

$$\min_{\theta} \ \mathbb{E}\left[\mathcal{L}_{\text{pred}}(\hat{\mathbf{Y}}, \mathbf{Y})\right]. \tag{1}$$

$$\hat{y}_{t+h} \in \mathcal{F}_{\text{phys}}(v_{t+h}^{\text{proxy}}), \qquad v_{t+h}^{\text{proxy}} = v_t, \quad h = 1, \ldots, H. \tag{2}$$

where $\mathcal{F}_{\text{phys}}(\cdot)$ denotes the feasible power envelope and v_{t+h}^{proxy} is the deployable wind proxy used throughout this paper. It is parameterized by turbine specifications (P_{rated}, v_{in}, v_{rated}, v_{out}). The construction of $\mathcal{F}_{\text{phys}}$ is given in Section **??**.

3.2. Multi-scale decomposition and embedding

The front-end separates slow trend variation from faster residual fluctuations before routing. Following decomposition–fusion practice in renewable forecasting [**? ?**], we apply a moving-average operator:

$$\mathbf{X} = \mathbf{X}_{\text{trend}} + \mathbf{X}_{\text{season}}, \qquad \mathbf{X}_{\text{trend}} = \text{MA}(\mathbf{X}), \tag{3}$$

where $\text{MA}(\cdot)$ is the moving-average operator.

The residual component is embedded through parallel 1D convolutions:

$$\mathbf{E}_s = \text{Conv1D}_{k_s}(\mathbf{X}_{\text{season}}), \tag{4}$$

$$\mathbf{E}_m = \text{Conv1D}_{k_m}(\mathbf{X}_{\text{season}}), \tag{5}$$

followed by learnable fusion:

$$\mathbf{E} = \phi(\mathbf{W}_s \mathbf{E}_s + \mathbf{W}_m \mathbf{E}_m), \tag{6}$$

where $\phi(\cdot)$ is a pointwise nonlinearity and $\mathbf{W}_s, \mathbf{W}_m$ are projection matrices. This decomposition-embedding block provides the input representation for sparse routing.

3.3. Sparse routing module

This module selects a small subset of experts for each token through Top-K routing. The formulation follows classical conditional computation and load-balanced gating, adapted here to time-series forecasting with a Jump Gate [**? ?**].

3.3.1. Routing formulation

Given the decomposition-and-embedding front-end above, the token-wise router input is

$$\mathbf{e}_t = \mathbf{E}[t, :], \tag{7}$$

where $\mathbf{E} \in \mathbb{R}^{L \times d}$ is the fused embedding from Section **??** and $\mathbf{e}_t \in \mathbb{R}^d$ is the representation routed at step t.

Figure **??** shows the model interface under the main Wind (CN) setting ($L = 96$, $H = 24$): decomposition, sparse Top-K routing, Jump Gate skipping, and the feasibility-guided training path used in the reported protocol.

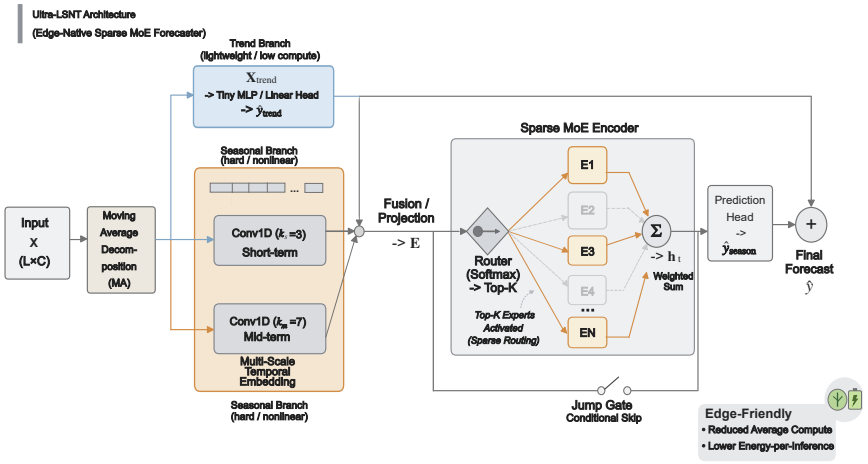


Figure 1: Ultra-LSNT architecture with decomposition, multi-scale Conv1D embedding, Top-K MoE routing, and a Jump Gate skip path together with the feasibility-guided training path used in this study.

Frequency-aware alternatives such as VMD/EEMD can be used in the same interface [**? ?**].

3.3.2. Training objective

Let $\mathbf{e}_t \in \mathbb{R}^d$ denote the embedding at step t. Let $\{E_i(\cdot)\}_{i=1}^N$ denote expert networks:

$$E_i(\mathbf{e}_t) = \sigma(\mathbf{W}_i \mathbf{e}_t + \mathbf{b}_i), \tag{8}$$

where $\mathbf{W}_i \in \mathbb{R}^{d \times d}$ and $\mathbf{b}_i \in \mathbb{R}^d$.

A router computes logits and probabilities:

$$\mathbf{z}_t = \mathbf{W}_r \mathbf{e}_t + \boldsymbol{\epsilon}_t, \qquad \mathbf{p}_t = \mathrm{Softmax}(\mathbf{z}_t), \tag{9}$$

where $\mathbf{W}_r \in \mathbb{R}^{N\times d}$ and $\boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \eta_e^2\mathbf{I})$. The exploration scale is annealed by epoch index e:

$$\eta_e = \eta_0 \exp(-e/\tau_\eta). \tag{10}$$

Top-K routing selects $\mathcal{S}_t = \mathrm{TopK}(\mathbf{p}_t, K)$ and aggregates expert outputs as

$$\mathbf{o}_t = \sum_{i\in\mathcal{S}_t} p_{t,i}\, E_i(\mathbf{e}_t). \tag{11}$$

Training uses prediction loss and routing regularization.

Point forecasting is optimized by MSE:

$$\mathcal{L}_{\text{pred}} = \frac{1}{H}\sum_{h=1}^{H} \|\hat{y}_{t+h} - y_{t+h}\|_2^2 . \tag{12}$$

Router entropy and expert disagreement are defined as

$$H_t = -\sum_{i=1}^{N} p_{t,i}\log(p_{t,i}+\epsilon), \tag{13}$$

with $\epsilon = 10^{-8}$, and

$$D_t = \frac{1}{|\mathcal{S}_t|}\sum_{i\in\mathcal{S}_t}\left(\hat{y}^{(i)}_{t+h} - \bar{y}_{t+h}\right)^2 . \tag{14}$$

$$\bar{y}_{t+h} = \frac{1}{|\mathcal{S}_t|}\sum_{i\in\mathcal{S}_t}\hat{y}^{(i)}_{t+h}. \tag{15}$$

These quantities parameterize an adaptive reserve rule:

$$\sigma_{w,t} = \min\left(\sigma_{\max}, \sigma_{\text{base}} + \alpha_H H_t + \alpha_D D_t\right), \tag{16}$$

where σ_{base}, α_H, α_D, and $\sigma_{\max}$ are fixed coefficients. Optional probabilistic heads are

$$\mathcal{L}_{\text{pin}} = \frac{1}{|\mathcal{Q}|H}\sum_{q\in\mathcal{Q}}\sum_{h=1}^{H}\rho_q\left(y_{t+h} - \hat{y}^{(q)}_{t+h}\right). \tag{17}$$

$$\rho_q(e) = \max(qe, (q-1)e). \tag{18}$$

$$\mathcal{L}_{\text{nll}} = \frac{1}{H}\sum_{h=1}^{H}\left(\frac{(y_{t+h}-\mu_{t+h})^2}{2\sigma_{t+h}^2} + \log\sigma_{t+h}\right). \tag{19}$$

The router load-balancing regularizer is

$$\mathcal{L}_{\text{aux}} = \sum_{i=1}^{N}\left(\bar{p}_i - \frac{1}{N}\right)^2, \qquad \bar{p}_i = \frac{1}{T}\sum_{t=1}^{T} p_{t,i}, \tag{20}$$

and the routing-module objective is

$$\mathcal{L}_{\text{base}} = \mathcal{L}_{\text{pred}} + \lambda_r\,\mathcal{L}_{\text{aux}}. \tag{21}$$

3.4. Feasibility-guided envelope regularization

FG-MoE adds a penalty that discourages forecasts outside the turbine envelope. The envelope is built from aerodynamic power conversion and a SCADA-observable proxy that remains available at deployment.

A horizontal-axis turbine is bounded by

$$P_{\text{aero}}(v) = \frac{1}{2}\rho A v^3 C_p(\lambda_{\text{tsr}}, \beta), \qquad A = \pi R^2, \tag{22}$$

where ρ is air density, A is rotor swept area, v is hub-height wind speed, and C_p is the power coefficient with $0 \le C_p \le C_{p,\max} \le 16/27$. The operational envelope is parameterized by $(P_{\text{rated}}, v_{\text{in}}, v_{\text{rated}}, v_{\text{out}})$ following standard wind-turbine power-curve conventions and test practice [**? ? ?**].

Because future wind-speed states are unavailable during deployment, FG-MoE uses the last observed hub-height wind speed and repeats it across the prediction horizon:

$$v_{t+h}^{\text{proxy}} = v_t, \qquad h = 1, \dots, H. \tag{23}$$

This choice matches the deployable proxy used in both training and evaluation, and avoids depending on unavailable future wind observations.

Using v_{t+h}^{proxy}, the envelope is

$$P_{\text{env}}(v^{\text{proxy}}) = \begin{cases} 0, & v^{\text{proxy}} < v_{\text{in}}, \\ P_{\text{rated}} \left(\frac{v^{\text{proxy}} - v_{\text{in}}}{v_{\text{rated}} - v_{\text{in}}} \right)^3, & v_{\text{in}} \le v^{\text{proxy}} < v_{\text{rated}}, \\ P_{\text{rated}}, & v_{\text{rated}} \le v^{\text{proxy}} \le v_{\text{out}}, \\ 0, & v^{\text{proxy}} > v_{\text{out}}. \end{cases} \tag{24}$$

Let $\tilde{y}_{t+h}^{\text{raw}} = \hat{y}_{t+h}/P_{\text{rated}}$, $\tilde{P}_{\text{env}}(v_{t+h}^{\text{proxy}}) = P_{\text{env}}(v_{t+h}^{\text{proxy}})/P_{\text{rated}}$, and let $\mathbb{I}_{\text{cut}}(v_{t+h}^{\text{proxy}}) = 1$ when $v_{t+h}^{\text{proxy}} < v_{\text{in}}$ or $v_{t+h}^{\text{proxy}} > v_{\text{out}}$. FG-MoE regularizes the raw forecast through

$$\begin{aligned} \mathcal{L}_{\text{physics}} = \frac{1}{H} \sum_{h=1}^{H} \Big[& \max(0, -\tilde{y}_{t+h}^{\text{raw}})^2 + \max(0, \tilde{y}_{t+h}^{\text{raw}} - 1)^2 \\ & + \max(0, \tilde{y}_{t+h}^{\text{raw}} - \tilde{P}_{\text{env}}(v_{t+h}^{\text{proxy}}))^2 \\ & + \mathbb{I}_{\text{cut}}(v_{t+h}^{\text{proxy}})(\tilde{y}_{t+h}^{\text{raw}})^2 \Big]. \end{aligned} \tag{25}$$

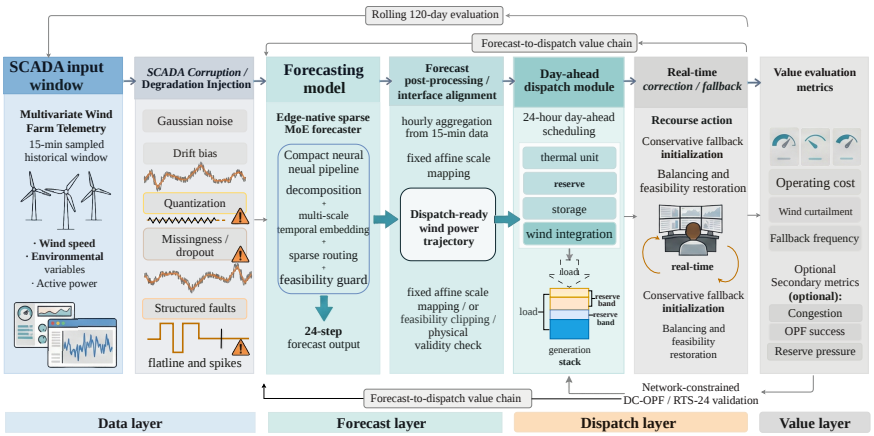


Figure 2: Fixed-interface dispatch screening pipeline from SCADA window to dispatch outcomes.

For the posthoc clip-only control and the fixed dispatch screening interface, we additionally use the deterministic projection

$$\tilde{y}^{\mathrm{clip}}_{t+h} = \mathrm{clip}\Big(\tilde{y}^{\mathrm{raw}}_{t+h},\ 0,\ \tilde{P}_{\mathrm{env}}(v^{\mathrm{proxy}}_{t+h})\Big), \tag{26}$$

which is not the core FG-MoE training path in the main protocol. Combining Eqs. (**??**), (**??**), and (**??**), the final objective is

$$\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{pred}}(\hat{\mathbf{Y}}^{\mathrm{raw}}, \mathbf{Y}) + \lambda_r \mathcal{L}_{\mathrm{aux}} + \lambda_2 \mathcal{L}_{\mathrm{physics}}. \tag{27}$$

For reproducibility, Algorithm **??** summarizes training with imperfection injection and feasibility guidance, Algorithm **??** summarizes edge inference with Jump Gate skipping, and Fig. **??** shows the workflow from SCADA measurements to dispatch screening outcomes.

Figure **??** shows the end-to-end dispatch-screening chain. The shared scale mapping and deterministic clipping belong to the screening interface, not to the FG-MoE training path used for the accuracy and safety results.

3.5. Conditional skipping for edge inference

The Jump Gate defines token-wise conditional skipping. The gate score is

$$I_t = \mathrm{Sigmoid}(\mathbf{w}_g^{\top} \mathbf{e}_t), \tag{28}$$

Algorithm 1 Ultra-LSNT training workflow with SCADA imperfection injection.

Input: training samples $(\mathbf{X}, \mathbf{Y})$ with $\mathbf{X} \in \mathbb{R}^{L \times C}$, $\mathbf{Y} \in \mathbb{R}^{H \times 1}$

Hyperparameters: experts N, Top-K, router regularizer λ_r, physics weight λ_2, noise config σ_{cfg}, industrial scale α, Jump Gate threshold τ

$\sigma_{\text{eff}} \leftarrow \alpha \cdot \sigma_{\text{cfg}}$

$\tilde{\mathbf{X}} \leftarrow \text{InjectImperfections}(\mathbf{X}; \sigma_{\text{cfg}}, \sigma_{\text{eff}})$

(Gaussian noise, drift, quantization; see Sec. **??** *for definitions.)*

$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\tilde{\mathbf{X}})$

$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$

for $b = 1$ **to** N_{blk} **do**

  $\mathbf{g} \leftarrow \text{Router}_b(\mathbf{Z})$ *(routing probabilities)*

  $\mathcal{E} \leftarrow \text{TopK}(\mathbf{g}, K)$ *(selected experts)*

  $\mathbf{Z} \leftarrow \text{AggregateExperts}_b(\mathbf{Z}, \mathcal{E})$

  $\mathbf{Z} \leftarrow \mathbf{Z} + \text{TrendInteract}_b(\mathbf{Z}, \mathbf{X}_{\text{trend}})$

end for

$\hat{\mathbf{Y}}^{\text{raw}} \leftarrow \text{ForecastHead}(\mathbf{Z})$

$\hat{\mathbf{V}}^{\text{proxy}} \leftarrow \text{RepeatLastWind}(\tilde{\mathbf{X}})$

$\mathcal{L}_{\text{pred}} \leftarrow \text{MSE}(\hat{\mathbf{Y}}^{\text{raw}}, \mathbf{Y})$

$\mathcal{L}_{\text{aux}} \leftarrow \text{LoadBalance}(\mathbf{g})$

$\mathcal{L}_{\text{physics}} \leftarrow \text{PhysicsPenalty}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}}^{\text{proxy}})$

Update parameters to minimize $\mathcal{L}_{\text{pred}} + \lambda_r \mathcal{L}_{\text{aux}} + \lambda_2 \mathcal{L}_{\text{physics}}$

with hard-threshold transition

$$\mathbf{o}_t = \begin{cases} \mathbf{o}_{t-1}, & I_t < \tau, \\ \text{MoE}(\mathbf{e}_t), & \text{otherwise}, \end{cases} \tag{29}$$

where τ is the threshold parameter.

During training, the hard gate is relaxed as

$$\tilde{z}_t = \sigma((I_t - \tau)/s), \qquad \mathbf{o}_t = \tilde{z}_t \cdot \text{MoE}(\mathbf{e}_t) + (1 - \tilde{z}_t) \cdot \mathbf{o}_{t-1},$$

where $s > 0$ is temperature.

Algorithm 2 Edge inference workflow with Jump Gate conditional skipping.

Input: window $\mathbf{X} \in \mathbb{R}^{L \times C}$
$(\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}) \leftarrow \text{Decompose}(\mathbf{X})$
$\mathbf{Z} \leftarrow \text{MultiScaleEmbed}([\mathbf{X}_{\text{trend}}, \mathbf{X}_{\text{resid}}])$
for $b = 1$ **to** N_{blk} **do**
  if $\text{JumpGate}_b(\mathbf{Z}; \tau) = \text{SKIP}$ **then**
    continue
  end if
  Route experts and aggregate as in Algorithm **??**
end for
$\hat{\mathbf{V}}^{\text{proxy}} \leftarrow \text{RepeatLastWind}(\mathbf{X})$
$\hat{\mathbf{Y}}^{\text{raw}} \leftarrow \text{ForecastHead}(\mathbf{Z})$
Optional screening: $\hat{\mathbf{Y}}^{\text{clip}} \leftarrow \text{PhysicsClip}(\hat{\mathbf{Y}}^{\text{raw}}, \hat{\mathbf{V}}^{\text{proxy}})$
Output: $\hat{\mathbf{Y}}^{\text{raw}}$ *(or* $\hat{\mathbf{Y}}^{\text{clip}}$ *for the clip-only control / dispatch interface)*

Ultra-LSNT-Lite is defined by reducing sparse-module capacity with Top-1 routing:

$$(N, K, \#\text{MoE blocks}, d) \;\rightarrow\; (2, 1, 1, 96), \tag{30}$$

where d is the hidden dimension.

Complexity follows directly. Let d denote hidden dimension. A dense N-expert computation scales as $O(N \cdot L \cdot d^2)$, whereas Top-K routing scales as $O(K \cdot L \cdot d^2)$ with $K \ll N$. If Jump Gate is enabled, the effective token count is multiplied by the non-skip ratio.

3.6. Evaluation metrics and robustness criteria

Primary point-forecasting metrics are R^2, RMSE, and MAE, computed after inverse scaling in the original physical units of each dataset and aggregated over the full horizon and test split. We retain MAPE for completeness, but because wind-power series contain frequent near-zero periods, ratio-based errors can be disproportionately inflated even with numerical safeguards; MAPE is therefore treated as a supplementary diagnostic rather than a headline metric.

$$R^2 = 1 - \frac{\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}{\sum_{t=1}^{T}(y_t - \bar{y})^2}. \tag{31}$$

$$\mathrm{RMSE} = \sqrt{\frac{1}{T}\sum_{t=1}^{T}(y_t - \hat{y}_t)^2}. \tag{32}$$

$$\mathrm{MAE} = \frac{1}{T}\sum_{t=1}^{T}|y_t - \hat{y}_t|. \tag{33}$$

$$\mathrm{MAPE} = \frac{100}{T}\sum_{t=1}^{T}\left|\frac{y_t - \hat{y}_t}{y_t + \epsilon}\right|. \tag{34}$$

Here $\bar{y}$ is the sample mean of y_t, and ϵ is a small constant used to avoid numerical instability when y_t approaches zero.

Average point errors do not fully characterize controller-side usefulness. We therefore audit tail-error, ramp, smoothness, and routing-stability indicators alongside the primary metrics, and for probabilistic variants we additionally report PICP, PINAW, and CRPS. Robustness tests combine continuous perturbations (Gaussian noise, drift, quantization) with structured SCADA faults (flatline, spikes, correlated dropouts). Throughout the main protocol, $\sigma_{\mathrm{eff}} = 0.15$ is treated as the empirical upper-band anchor, while $\sigma_{\mathrm{eff}} \in \{0.30, 0.45, 0.60\}$ defines an explicit stress envelope.

4. Experimental setup and robustness protocol

The experiments use one shared pipeline for data preparation, benchmark selection, corruption stress, and dispatch-coupled screening.

4.1. Data, preprocessing, and tasks

The primary case study is ultra-short-term wind-power forecasting using SCADA measurements from a utility-scale wind farm in Xinjiang, China. The setup is intended to reflect operational conditions with sensor uncertainty and non-stationary weather. Inputs include wind and environmental variables, and the target is aggregated active power.

Table 1: Dataset characteristics and SCADA degradation protocol.

Item	Specification
	Panel A
	Multi-domain dataset spatiotemporal characteristics
Wind (CN) SCADA (primary)	15-min sampling, 121,528 records, train/test chronology 80%/20% (97,222/24,306), train range 2021-04-02 00:00 to 2022-03-12 15:45, test range 2022-03-12 16:00 to 2022-06-30 23:45, 8 input channels, target active power in kW.

Table 1: Dataset characteristics and SCADA degradation protocol (continued).

Item	Specification
Wind (US) transfer	5-min sampling, 105,120 records, full-year range 2012-01-01 00:00 to 2012-12-31 23:55, 10 input channels (time/covariates) with wind-power target in MW.
Air Quality transfer	Daily city-panel records, 588,644 samples after preprocessing, range 2015-01-01 to 2023-05-31, 7 pollutant/air-quality channels (AQI, PM2.5, PM10, SO_2, NO_2, CO, O_3).
GEFCom Load transfer	1-hour sampling, 38,070 records, range 2004-01-01 00:00:00 to 2008-06-30 05:00:00, univariate load target with calendar-aligned chronological split.
Unified task protocol	All datasets use chronological splitting, train-fitted z-score normalization, inverse scaling before metric reporting, and common QC (duplicate removal, missing-value handling, NaN/Inf sanity checks). Main forecasting setup uses lookback/horizon $(L, H) = (96, 24)$.
Panel B	
SCADA degradation and stress protocol (Wind CN main benchmark)	
Gaussian noise (inference-time)	Configured severity $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ with industrial scaling $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, $\alpha = 1.5$, giving effective tiers $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$.
Drift bias (random-walk)	Persistent bias injected as a channel-wise random walk with step scale $\sigma_{\text{eff}} \cdot \text{std}(x)$; this emulates calibration drift and low-frequency sensor offset accumulation.
Quantization noise	Deterministic rounding with step size $\Delta = \sigma_{\text{eff}} \cdot \text{std}(x)$, i.e., $x_t^{\text{quant}} = \Delta \cdot \text{round}(x_t/\Delta)$, to emulate low-precision SCADA/edge telemetry.
Structured SCADA faults	Flatline, spike/heavy-tail outlier, and correlated dropout with contiguous windows; severity levels $\{0.2, 0.6\}$, with level 0.6 used as the severe-certification slice in main robustness tables.
Missingness and interpolation	i.i.d. masking rate $p \in \{0.1, 0.3\}$ with paired `ffill`/linear interpolation (3 seeds, shared masks across models); burst/correlated missingness is represented by structured dropout faults.
Empirical severity anchor	Channel-wise diff-proxy calibration on Wind (CN) yields median 0.045, P90 0.091, max 0.121; therefore $\sigma_{\text{eff}} = 0.15$ is treated as empirical upper-band neighborhood, while $\{0.30, 0.45, 0.60\}$ are explicit stress envelopes.
Dispatch coupling protocol	For dispatch, 15-min Wind (CN) is aggregated to 1-hour resolution; 120-day rolling copper-plate and IEEE RTS-24 DC-OPF screening are both reported under fixed affine scale mapping and fixed feasibility clipping rules.

Table **??** gathers dataset scope, stress tiers ($\sigma_{\text{eff}} \in \{0.15, 0.30, 0.45, 0.60\}$), and dispatch aggregation (15-min to hourly) in one place, so the main comparisons do not shift preprocessing or severity settings from model to model. Figure **??** shows that the study covers heterogeneous temporal patterns rather than a single profile. To test transfer behavior, we include three additional tasks: *Wind (US)*, *Air Quality*, and *GEFCom Load*. All datasets are split chronologically to prevent leakage. Z-score normalization is fitted on the training split only and inverse scaling is applied before reporting physical-unit metrics. The same quality-control pipeline (duplicate

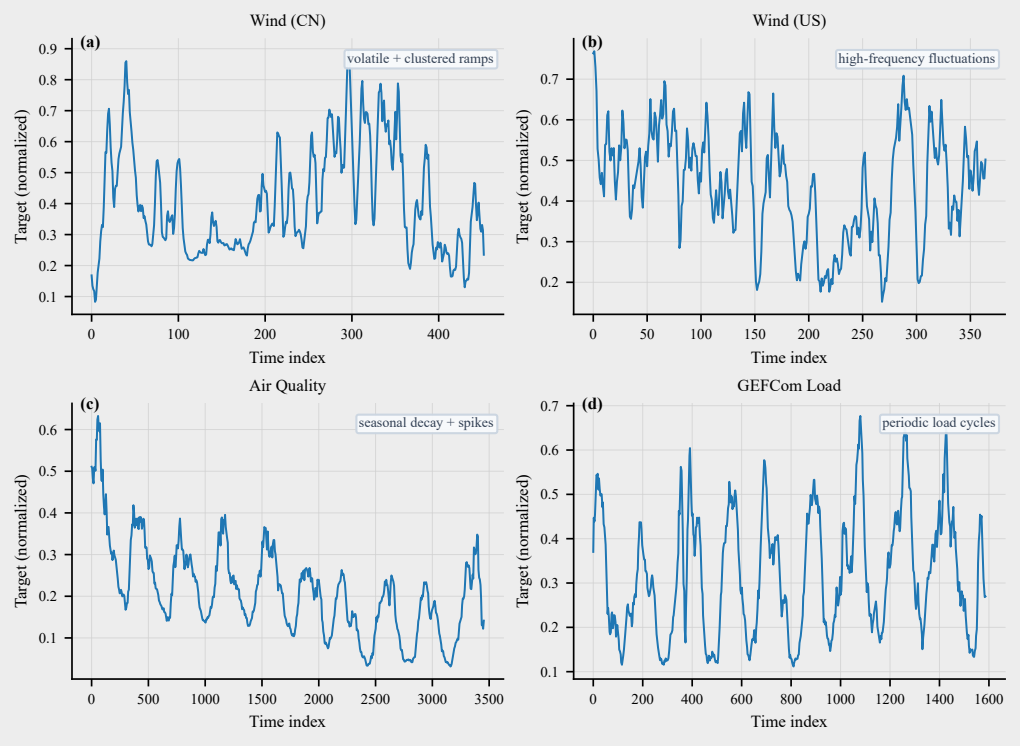


Figure 3: Representative raw-series excerpts for Wind (CN), Wind (US), Air Quality, and GEFCom Load. Series are min-max normalized within each dataset for visual comparability and plotted with dataset-specific excerpt length.

removal, missing-value handling, NaN/Inf checks) is applied across all experiments.

4.2. Baselines and protocol design

Primary point metrics are defined in Section **??**; here the comparison is kept on one common footing. The main robustness evidence uses two views of the same benchmark. Panel A of Table **??** places eight models (DLinear, TimeMixer, iTransformer, Ultra-LSNT, LightGBM, Persistence, Chronos-Bolt-Small, Moirai-2.0-R-Small) under the same 24-step target window and identical test-time corruption, so the severe-slice ranking can be seen in context. Panel B keeps only DLinear, TimeMixer, iTransformer, and Ultra-LSNT, each with clean-trained and fault-augmented counterparts (three seeds; four faults: Gaussian, drift, quantization, missingness), so robustness changes can be compared without changing model family and training scope at the same time.

At the severe Gaussian slice ($\sigma_{\text{eff}} = 0.60$), fault-augmented training changes mean R^2 from

0.2402 to 0.3471 for DLinear, 0.7452 to 0.8417 for TimeMixer, 0.8478 to 0.8580 for iTransformer, and 0.8135 to 0.8180 for Ultra-LSNT. The broader ranking remains in Panel A; the training-effect comparison sits in Panel B, where the model family is held fixed.

Chronos and Moirai are retained in Panel A as long-context reference models under zero-shot extended-context usage. They are not included in controller-side latency, memory, thermal, PLC/RTU, or batch-1 deployability claims. Long-tail baselines, including decomposition-optimization hybrids and constrained-budget metaheuristic probes, are moved to Appendix and the archived supplementary evidence bundle rather than the main matched-control comparison.

LightGBM and Persistence. To keep the non-deep baselines aligned with the deep models, we use the same chronological split, window sizes ($L = 96$, $H = 24$), train-fitted z-score normalization with inverse scaling, and identical test-time corruption settings. Gaussian stress uses the same effective-noise rule $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$ with $\alpha = 1.5$. LightGBM is implemented as `MultiOutputRegressor(LGBMRegressor)` on flattened historical windows using only $x_{t-L+1:t}$ (no future covariates), with fixed hyperparameters (`n_estimators`=500, `learning_rate`=0.05, `random_state`=42, `n_jobs`=-1) to avoid asymmetric tuning budgets. Persistence repeats the last observed target over the full prediction horizon.

Beyond the primary point metrics, we audit tail-error, ramp, smoothness, and routing-stability indicators under the same inverse-scaling protocol; for probabilistic variants we additionally report PICP, PINAW, and CRPS. Robustness tests include continuous perturbations (Gaussian noise, drift, quantization) and structured faults (flatline, spikes, correlated dropouts), with Gaussian stress up to 60% of clean standard deviation and structured-fault severities $\{0.2, 0.6\}$. Full perturbation definitions are provided in Appendix **??**. We also report latency, model size, parameter count, and an analytical energy proxy under fixed profiling settings [**?** **?**].

4.3. Dispatch-coupling and screening interface

Dispatch outcomes depend jointly on the predictor, the forecast-to-grid interface, and the dispatch rule. To keep the latter two fixed across models, we use one archived mapped-and-clipped interface for all forecasts and interpret the resulting numbers as screening results under that shared interface. Broader mapping sensitivity is retained in the supplementary material.

For operational coupling, we formulate day-ahead (DA) scheduling on a 24-hour horizon with 1-hour resolution [**?**]. Forecast impact is summarized by three indicators: (i) operating cost, (ii) wind curtailment, and (iii) feasibility-restoration (fallback) frequency. Monetary outcomes

are reported in CNY. We also run a two-stage DA/RT DC-OPF supplement on IEEE RTS-24 and report total slack, congestion indicators, and OPF success ratio when available. Both the copper-plate DA engine and the RTS-24 DC-OPF layer are screening models rather than operator certification tools; the latter captures active-power congestion effects under linearized network physics and is not an AC-feasibility guarantee [**?**].

To quantify forecast value in dispatch, we couple each model output to a simplified DA engine with one thermal unit, upward reserve procurement, and storage [**?** **?**]. Let $\hat{w}_t$ denote forecast wind power (kW), d_t demand (kW), and let dispatch variables be thermal output g_t (kW), reserve r_t (kW), curtailment u_t (kW), storage charge/discharge powers $p_{\mathrm{ch},t}$, $p_{\mathrm{dis},t}$ (kW), and state-of-charge s_t (kWh). The dispatch grid is hourly. We therefore aggregate Wind (CN) from 15-min to 1-hour resolution by averaging each 4 consecutive records (Table **??**).

Ultra-LSNT is trained on this hourly series with $L = 96, H = 24$ and outputs $\hat{w}_{1:24}$ directly for dispatch input. With $\Delta t = 1\,\mathrm{h}$ and horizon $T = 24$, every forecaster provides a 24-step daily vector $\hat{w}_{1:T}$. We evaluate a 120-day rolling window for Ultra-LSNT, iTransformer, TimeMixer, and DLinear. A day is tagged as `fallback_x0` when the optimizer needs conservative fallback initialization under unchanged algebraic constraints.

To obtain comparable dispatch screening inputs, we apply one archived global mapping followed by deterministic clipping before DA/RT optimization. This mapped-and-clipped interface is fixed across models and is not retuned per model.

Raw and mapped outcomes are both reported in Table **??**. In raw-scale stress checks, all models stay in `fallback_x0` for all 120 days, with optimized ratio 0. Under the shared interface, optimized ratios become non-zero and the resulting metrics are finite across models.

To keep operating bounds consistent across models in rolling runs, we apply train-split bound clipping to each daily forecast vector:

$$\tilde{w}_t = \mathrm{clip}(s\hat{w}_t, 0, w_{\max}), \tag{35}$$

$$s = \min\left(1, \frac{w_{\max}}{\max_t \hat{w}_t + \epsilon}\right), \tag{36}$$

where $w_{\max} = \max(w_{\mathrm{train}})$ is computed on the hourly training split and ϵ is a small constant. Results under this mapping are reported in Table **??**, and both raw/mapped rolling summaries are included in supplementary evidence.

All monetary coefficients are in CNY. Because absolute totals depend on price coefficients

and reference year, we report both absolute values and relative reductions against fixed baselines. With $\Delta t = 1\,\mathrm{h}$, summed kW schedules correspond to kWh energy over the horizon. Dispatch metrics are always computed against realized wind output on the same test day for all models.

We include one UQ-aware reserve variant where $\sigma_{w,t}$ scales with routing entropy and expert disagreement from the MoE forward pass. For comparability, we also report the fixed-proportional reserve rule $\sigma_{w,t} = 0.2|\hat{w}_t|$.

Note on network constraints. The dispatch model above is single-node (copper-plate) and isolates how forecast errors propagate to reserve, curtailment, and operating cost. We additionally run network-constrained DC-OPF on IEEE RTS-24 as a comparative screening layer. DC-feasible schedules are not guaranteed to be AC-feasible.

All experiments run in one pipeline with fixed preprocessing, chronological splitting, and consistent metric computation. Full environment/training settings are reported in **??**. Source code is released at `https://github.com/b1ue13e/Ultra-LSNT` (MIT), and Wind (CN) data plus preprocessing/evaluation scripts are released for end-to-end reproduction.

5. Results

This section reports four result groups: robustness under SCADA corruption, hardware and safety trade-offs, dispatch screening under a shared interface, and operating-point diagnostics in (K, τ, λ_2) space.

5.1. Robustness under SCADA corruption

Table **??** separates a context panel from matched-control evidence. In Panel A, the long-context reference models remain highest at the severe Gaussian slice ($R^2 = 0.8875$ for Moirai-2.0-R-Small and 0.8792 for Chronos-Bolt-Small), iTransformer and Persistence follow (0.8717 and 0.8612), Ultra-LSNT remains positive at 0.7863, and DLinear falls to -0.9903. Because Chronos and Moirai are reported only as zero-shot extended-context references, this panel should be read as ranking context under a shared corruption protocol rather than as an edge-deployment comparison.

Panel B holds architecture fixed and changes only the training condition. Switching from clean-trained to fault-augmented training improves the severe-slice mean R^2 from 0.2402 to 0.3471 for DLinear, 0.7452 to 0.8417 for TimeMixer, 0.8478 to 0.8580 for iTransformer, and

Table 2: Audited robustness evidence on Wind (CN).

Panel A

Clean-trained context panel (8 models; edge-feasible models + reference models)

Model	Clean R^2	Clean RMSE	R^2@0.60	RMSE@0.60	Rel. RMSE inc. (%)
DLinear	0.9494	7406.87	-0.9903	46449.77	527.12
TimeMixer	0.8886	8498.38	0.8337	10382.59	22.17
iTransformer	0.9012	8004.05	0.8717	9117.01	13.90
Ultra-LSNT	0.9346	8419.91	0.7863	15218.84	80.75
LightGBM	0.9563	6880.91	0.7121	17665.50	156.73
Persistence	0.9175	7313.94	0.8612	9485.55	29.69
Chronos-Bolt-Small	0.9644	6216.19	0.8792	11443.18	84.09
Moirai-2.0-R-Small	0.9686	5834.04	0.8875	11045.80	89.33

Panel B

Matched-control severe-slice panel

Model	Clean-trained R^2@0.60	Fault-augmented R^2@0.60	Gain
DLinear	0.2402	0.3471	+0.1069
TimeMixer	0.7452	0.8417	+0.0965
iTransformer	0.8478	0.8580	+0.0102
Ultra-LSNT	0.8135	0.8180	+0.0045

Note: Panel A provides an eight-model context view at clean and severe Gaussian slices. RMSE is retained in the main table and MAE is moved to Appendix **??** for completeness. The realistic operating anchor is $\sigma_{\text{eff}} = 0.15$, whereas $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ defines the stress envelope. Chronos and Moirai are reported as long-context reference models under zero-shot extended-context usage (512-step input context, same 24-step target window and evaluation split) rather than as edge-feasible controller-side models. Detailed audit-file manifests are provided in the supplementary evidence bundle.

0.8135 to 0.8180 for Ultra-LSNT. The largest increments appear in the weaker baselines, whereas Ultra-LSNT begins from a more stable corrupted-input controller-side baseline and changes only modestly under augmentation.

At the empirical anchor $\sigma_{\text{eff}} = 0.15$, archived intermediate-slice evidence for the six local context baselines remains in positive-skill territory (DLinear 0.7814, LightGBM 0.9340, Persistence 0.9140, Ultra-LSNT 0.8687, TimeMixer 0.8847, iTransformer 0.8993), consistent with the calibration reported in Appendix **??** (median 0.045, P90 0.091, max 0.121). The main tables therefore use $\sigma_{\text{eff}} = 0.15$ as the empirical anchor and $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ as explicit stress levels.

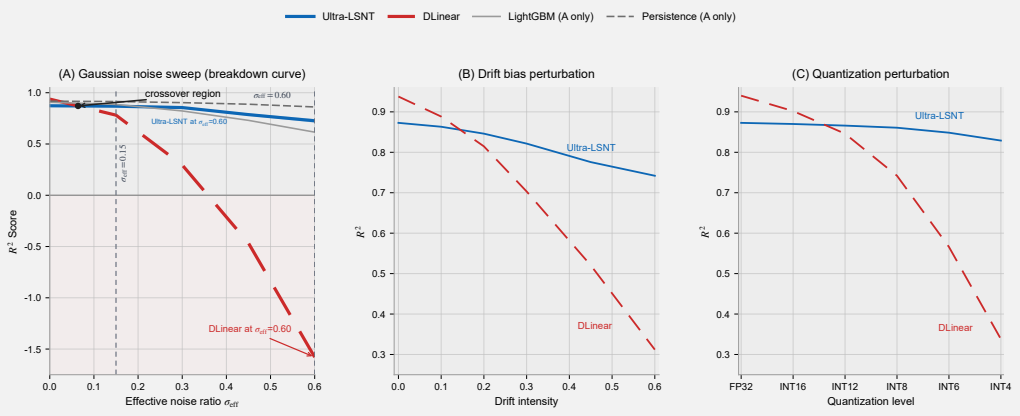


Figure 4: Robustness summary under Gaussian noise, drift bias, and quantization perturbations on Wind (CN). Panel A reports the full Gaussian sweep ($\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$) for Ultra-LSNT, DLinear, LightGBM, and Persistence, with $\sigma_{\text{eff}} = 0.15$ as the empirical upper-band anchor and $\sigma_{\text{eff}} = 0.60$ as the severe stress-envelope anchor. Panels B–C report drift and quantization trajectories for Ultra-LSNT and DLinear under the same protocol.

Figure **??** extends the same comparison across Gaussian, drift, and quantization trajectories. The full-σ sweep reinforces the same point as Table **??**: corrupted-operation robustness should be read from the degradation path, not inferred from the clean slice alone.

5.2. Hardware and safety trade-offs

Table **??** separates the edge-profile question from the feasibility question. Panel A shows that Ultra-LSNT is not the smallest model in the table, but its latency and memory remain within controller-side screening range while preserving sparse routing. Panel B compares no guardrail, clip-only repair, and FG-MoE under the same backbone. Clip-only drives exceedance to zero only by pushing clean and severe-slice R^2 down to 0.4319 and 0.3120, whereas FG-MoE keeps them at 0.8747 and 0.8286 while lowering exceedance to 0.0092%. The contrast is consistent with recent trustworthy physics-guided forecasting arguments that training constraints should reshape the learned solution rather than repair violations only after inference [**?**].

5.3. Dispatch screening results

Table **??** separates the interface problem from the mapped comparison. In Panel A, every raw-scale run remains in `fallback_x0` for all 120 days, so raw forecasts do not yield a usable

Table 3: Hardware profile and safety diagnostics under protocol P1.

Panel A

Batch-1 edge profile (CPU/ARM)

Model	CPU Latency (ms)	ARM Latency (ms)	CPU Throughput (Hz)	Memory (MiB)
Ultra-LSNT (full)	3.714	8.795	269.3	22.2830
Ultra-LSNT-Lite (opt.)	1.171	5.007	854.0	1.3327
DLinear	0.265	0.102	3767.6	0.0178
Mamba (logged) (bf)	0.861	0.861	1161.1	0.1468
VMD-PSO-LSTM (bf)	10.297	10.297	97.1	0.2468
CEEMDAN-WOA-GRU (bf)	11.082	11.082	90.2	0.2130
ARIMA (logged) (bf)	2.183	2.183	458.0	0.0000
SVR (logged) (bf)	2.750	2.750	363.6	0.5410
SSA-ELM (bf)	0.024	0.024	42409.6	0.8107

Panel B

Guardrail operating-point comparison

Setting	Clean R^2	R^2@0.60	Rated-power exceedance (%)
Ultra-LSNT (no guardrail)	0.8727	0.8273	0.0550
Ultra-LSNT + clip-only (posthoc)	0.4319	0.3120	0.0000
FG-MoE (full-trained)	0.8747	0.8286	0.0092

Note: Panel A reports direct CPU measurements together with archived ARM backfill used for cross-ISA screening. Panel B reports the protocol-P1 comparison among no guardrail, posthoc clip-only repair, and FG-MoE. Additional cut-region and envelope-deviation diagnostics are provided in the supplementary material.

comparative dispatch result. Panels B–C report the same models only after the shared dispatch-screening interface has been imposed, which is the setting in which dispatch metrics become comparable.

Under the shared interface, iTransformer gives the lowest L1 cost and fallback rate, TimeMixer follows, and Ultra-LSNT still improves L1 cost, curtailment, and fallback frequency relative to DLinear without becoming the dispatch leader in that table. Panel C adds a narrower network-constrained screen: all models remain DC-feasible on RTS-24, and the remaining difference collapses to curtailment improvement against the same base case. That pattern matches the broader forecast-value literature: dispatch benefits appear only after the scheduling model and interface are fixed, not from accuracy scores alone [**? ? ?**]. Under that fixed interface, the numbers support shared-interface comparative value for a balanced controller-side design rather

Table 4: Dispatch outcomes under 120-day rolling and RTS-24 DC-OPF evaluation.

Panel A

Fixed-interface comparability / fallback gate

Model	Raw opt.	Raw FB (%)	Mapped opt.	Mapped FB (%)
DLinear	0.0000	100.0	0.3750	62.5
TimeMixer	0.0000	100.0	0.7833	21.7
iTransformer	0.0000	100.0	0.8333	16.7
Ultra-LSNT	0.0000	100.0	0.7583	24.2

Panel B

L1 mapped-protocol outcomes (copper-plate)

Model	L1 Cost (M CNY)	L1 Curt. (M kWh)	L1 FB (%)
iTransformer	32.16	15.22	16.7
TimeMixer	34.90	19.60	21.7
Ultra-LSNT	35.97	22.09	24.2
DLinear (base)	54.47	55.71	62.5

Panel C

L2 mapped-protocol outcomes (RTS-24 DC-OPF)

Model	L2 Curt. (MWh)	L2 Succ. (%)	L2 Gain (%)
iTransformer	138,093.39	100.00	+0.94
TimeMixer	138,093.39	100.00	+0.94
Ultra-LSNT	138,093.39	100.00	+0.94
DLinear (base)	139,403.69	100.00	+0.00

Note: Panel A reports raw-vs-mapped optimization and fallback rates. Panels B and C report 120-day copper-plate and RTS-24 DC-OPF screening outcomes under that same archived interface. Broader mapping-sensitivity evidence is moved to the supplementary material.

than AC-feasible or operator-grade dominance.

5.4. Hyperparameter Pareto and operating points

Table **??** quantifies the accuracy-safety Pareto in (K, τ, λ_2) space under protocol P2 (joint-sensitivity SOA scan with router overrides, λ_2 post-guard correction, and partial-batch evaluation). This is an operating diagnostic rather than a new comparative leaderboard: low λ_2 keeps severe-noise R^2 near 0.93 with non-zero rated-power exceedance, while $\lambda_2 = 1.0$ removes that metric at large accuracy cost. Figure **??** shows the same protocol-bound trade-off at the anchor

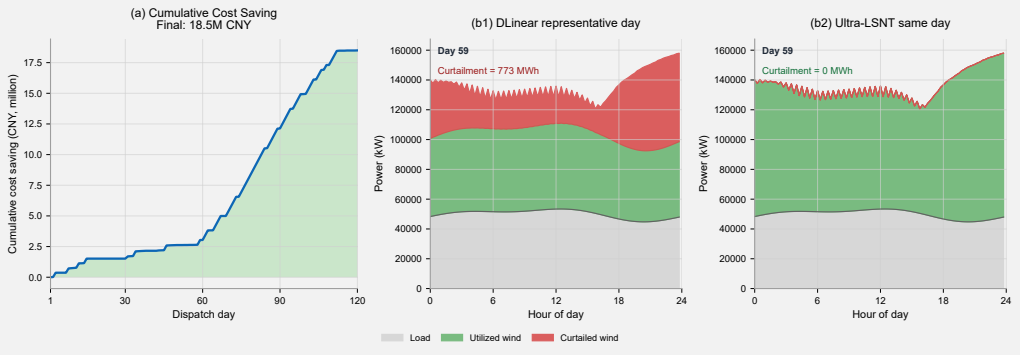


Figure 5: Dispatch screening under the shared mapped-and-clipped interface. Panel (a) shows 120-day cumulative cost saving against DLinear. Panels (b1) and (b2) show the representative day for DLinear and Ultra-LSNT, respectively.

Table 5: Pareto operating points in (K, τ, λ_2) at $\sigma_{\text{eff}} = 0.60$ under protocol P2 (SOA proxy).

K	τ	λ_2	**Latency (ms)**	R^2@0.60	**Rated-power exceed. (SOA proxy, %)**
1	0.7	0.01	10.609	0.9301	19.1332
1	0.9	0.10	10.232	0.9257	18.0169
2	0.5	0.01	14.353	0.9294	19.0091
4	0.9	0.01	19.769	0.9299	18.9690
1	0.9	1.00	10.232	0.3831	0.0000
4	0.5	1.00	18.182	0.3913	0.0000

Note: Values are sampled from the archived P2 sensitivity sweep. The reported safety axis is rated-power exceedance under the protocol-P2 SOA proxy and should not be numerically compared with the protocol-P1 values in Table **??**.

setting $(N, K, \tau) = (8, 2, 0.5)$ and should be used to select operating points within P2, not to compare directly against the protocol-P1 safety values in Table **??**.

6. Discussion and limitations

6.1. Why sparse routing helps under corrupted SCADA

SCADA corruption is heterogeneous over time: missing segments, change points, and failure-related drifts are concentrated rather than uniform [**?** **?** **?** **?**]. In that setting, a dense always-on pathway allocates the same computation to stable and unstable intervals. Ultra-LSNT instead lets harder windows activate more capacity while easier windows traverse a smaller path. Table **??** supports this interpretation only indirectly: under matched-control training, all models benefit from fault augmentation, but Ultra-LSNT begins from a higher severe-slice baseline

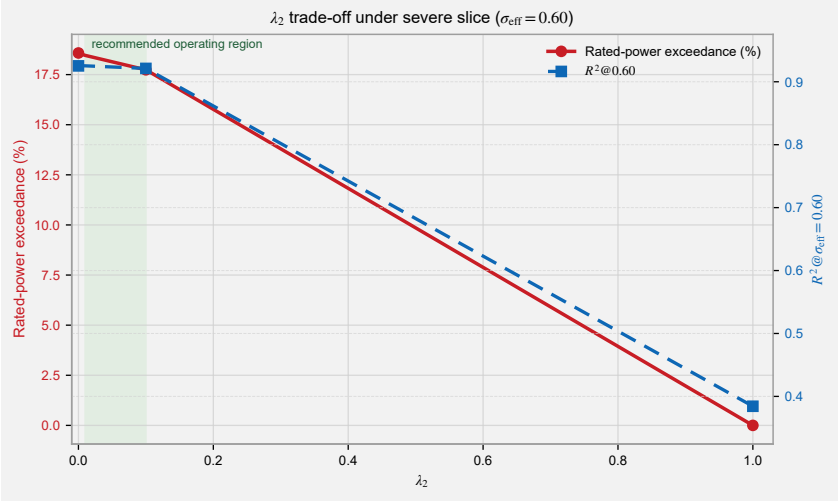


Figure 6: λ_2 safety-accuracy trade-off at anchor $(N, K, \tau) = (8, 2, 0.5)$ under the severe slice $\sigma_{\text{eff}} = 0.60$. The left axis reports rated-power exceedance ($\hat{y} > P_{\text{rated}}$, %) under protocol P2, and the right axis reports R^2@0.60. The shaded band indicates the recommended operating region.

and stays positive across the stress trajectory. That pattern is consistent with corruption-aware capacity allocation, although it is not a causal proof of the routing mechanism by itself.

6.2. *Why FG-MoE changes the forecast distribution*

FG-MoE differs from clip-only repair because the envelope enters training, not only the output projection. Recent wind power-curve preprocessing and physics-guided forecasting studies use physical priors to reshape learning before inference [**? ? ?**]. Table **??** makes the comparison concrete: clip-only removes exceedance but cuts clean and severe-slice R^2 to 0.4319 and 0.3120, whereas FG-MoE keeps 0.8747 and 0.8286 with exceedance at 0.0092%. The relevant distinction is therefore not whether outputs can be clipped after prediction, but whether training shifts the forecast distribution toward feasible operating regions without sacrificing the main accuracy regime.

6.3. *Dispatch screening, not operator validation*

Dispatch value remains interface-dependent. Coordinated scheduling and economic-value studies already show that forecast quality maps to different reserve, curtailment, and cost outcomes under different market or system models [**? ? ?**]. That is why the paper keeps one shared

dispatch-screening interface and one common workflow. The raw-scale closure result is part of that argument: without the shared interface, all 120-day runs stay fallback-only. Within the fixed interface, iTransformer has the lowest L1 cost and fallback rate, while Ultra-LSNT still lowers L1 cost and fallback days relative to DLinear; outside that interface, the manuscript makes no interface-free or table-leading dispatch claim.

6.4. External validity and remaining limits

Several limits remain explicit. The main forecasting evidence comes from one operational wind-farm dataset, so site-to-site heterogeneity is not exhausted. The dispatch module is a comparative screen rather than an operator certification tool, and DC feasibility on RTS-24 does not imply AC feasibility [**?**]. The hardware profile is based on direct CPU measurement plus archived ARM backfill, not board-level HIL measurement of latency, power, or thermal behavior; controller-side SCADA infrastructure studies remain broader than the measurement envelope reported here [**?**]. Chronos and Moirai remain useful long-context references in Panel A, but they are not edge-feasible controller baselines, and recent trustworthy or explainable wind-forecasting studies reinforce the need to separate benchmark context from deployment claims [**?** **?**]. P1 safety values in Table **??** and P2 SOA-proxy values in Table **??** come from different protocols and should not be compared directly.

7. Conclusion and outlook

This study addresses short-term wind forecasting at the intersection of corrupted SCADA inputs, batch-1 edge budgets, and deployable feasibility constraints. On Wind (CN), the evidence supports a sparse controller-side design that stays positive under severe corruption, uses FG-MoE to reduce rated-power violations without the accuracy collapse seen in clip-only repair, and remains competitive under the shared dispatch-screening interface without topping either the severe-context reference panel or the fixed-interface dispatch screen. The next step is not a broader benchmark claim but tighter external validation through additional sites, operator-grade AC dispatch studies, and board-level HIL measurement of latency, power, and thermal behavior.

CRediT authorship contribution statement

Junyu Li: Conceptualization, Methodology, Software, Validation, Formal analysis, Data curation, Writing – original draft, Visualization. **Juntao Du**: Supervision, Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Wind (CN): The Wind (CN) SCADA dataset used in this study is released publicly to support full reproducibility. The release includes the raw time series, the train/validation/test split indices, and the preprocessing scripts used to construct the learning task described in Table **??**. The dataset package is distributed via the project repository (`https://github.com/b1ue13e/Ultra-LSNT`) and will be mirrored to a long-term archival service at acceptance.

Public benchmarks: The additional datasets used for cross-domain evaluation are publicly accessible: (i) Wind (US) from the NREL Wind Toolkit [**?**], (ii) Air Quality from the UCI repository [**?**], and (iii) GEFCom load from the Global Energy Forecasting Competition (GEFCom2012) [**?**]. We report the exact dataset identifiers, preprocessing scripts, and evaluation protocols in the released code.

In line with Elsevier's research data practices, we provide reproducibility assets (protocol definitions, supplementary evidence tables, and run manifests) through the project repository and archived supplementary evidence bundle.

Code: The implementation, training scripts, and evaluation protocols are available at `https://github.com/b1ue13e/Ultra-LSNT` (MIT license).

Appendix A. Nomenclature and abbreviations

Item	Description
Abbreviations	
MoE	Mixture-of-Experts
SCADA	Supervisory Control and Data Acquisition
TSFM	Time-series foundation model
SOTA	State-of-the-art
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
FG-MoE	Feasibility-Guided MoE (main-text term in this manuscript)
UQ	Uncertainty Quantification
CRPS	Continuous Ranked Probability Score
AC-OPF	AC Optimal Power Flow
DC-OPF	DC Optimal Power Flow
HIL	Hardware-in-the-Loop
Symbols	
L	input length (time steps)
H	prediction horizon (time steps)
C	number of input variables/covariates
$\mathbf{X} \in \mathbb{R}^{L \times C}$	input sequence
$\hat{\mathbf{Y}} \in \mathbb{R}^{H \times 1}$	forecast output (wind power / target variable)
σ_{cfg}	configured perturbation ratio in robustness tests (dimensionless)
σ_{eff}	effective perturbation ratio, $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$
α	scaling factor used to map configured to effective noise severity ($\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, default $\alpha = 1.5$)
K	number of selected experts in Top-K routing
λ_r	load-balancing regularization coefficient for routing
τ	Jump Gate threshold for conditional skipping
R^2	coefficient of determination (scale-free)
RMSE/MAE	absolute errors in original physical units after inverse scaling
E_{inf}	estimated energy per sample inference (mJ)
B	batch size used in profiling (dimensionless)
N_{call}	number of forecasting calls per day
N_{turb}	number of turbines in a wind farm or fleet
c_f	fuel / purchase price coefficient in dispatch (CNY/kWh)
c_r	reserve price coefficient (CNY/kWh)
c_s	storage discharge cost coefficient (CNY/kWh)
c_{su}	start-up / shut-down cost coefficient (CNY)
$P_{\text{ch,max}}$	storage charging power limit (kW)
$P_{\text{dis,max}}$	storage discharging power limit (kW)
S_{max}	storage energy capacity limit (kWh)
R_{max}	thermal ramp-rate limit (kW per hour)
$\bar{P}_{\text{dev}}$	average device power during inference (W)

Item	Description
ρ	air density (kg/m^3), estimated from temperature/pressure
A	rotor swept area (m^2)
v	wind speed at hub height (m/s)
v_{t+h}^{proxy}	deployable wind proxy used by FG-MoE, with $v_{t+h}^{\text{proxy}} = v_t$
$C_p(\lambda_{\text{tsr}}, \beta)$	power coefficient (Betz-limited)
λ_{tsr}	tip-speed ratio (dimensionless)
β	pitch angle (deg)
P_{rated}	rated-power upper bound (kW)
$P_{\text{env}}(v_{t+h}^{\text{proxy}})$	turbine envelope evaluated at the deployable wind proxy (kW)
$v_{\text{in}}, v_{\text{out}}$	cut-in / cut-out wind speeds (m/s)
λ_2	plausibility-penalty coefficient in heuristic FG-MoE (dimensionless)

Appendix B. Dispatch model details

This appendix provides the deterministic dispatch algebra used by the screening workflow in Section **??**. The formulation is included for reproducibility and serves as comparative screening infrastructure rather than AC-secure operational guidance.

With horizon $T = 24$ and $\Delta t = 1$ h, the objective is

$$\min \; C_{\text{dispatch}} = C_{\text{fuel}} + C_{\text{reserve}} + C_{\text{purchase}} + C_{\text{storage}} + C_{\text{startup}} + C_{\text{shutdown}}, \tag{B.1}$$

$$C_{\text{fuel}} = c_f \sum_{t=1}^{T} g_t \Delta t,$$

$$C_{\text{reserve}} = c_r \sum_{t=1}^{T} r_t \Delta t,$$

$$C_{\text{purchase}} = c_f \sum_{t=1}^{T} p_{\text{ch},t} \Delta t,$$

$$C_{\text{storage}} = c_s \sum_{t=1}^{T} p_{\text{dis},t} \Delta t,$$

$$C_{\text{startup}} = c_{\text{su}} \sum_{t=2}^{T} \frac{\max(g_t - g_{t-1}, 0)}{R_{\max}},$$

$$C_{\text{shutdown}} = c_{\text{su}} \sum_{t=2}^{T} \frac{\max(g_{t-1} - g_t, 0)}{R_{\max}}.$$

Subject to

$$g_t + \hat{w}_t + p_{\mathrm{dis},t} - p_{\mathrm{ch},t} - u_t = d_t, \tag{B.2}$$

$$0 \le u_t \le \hat{w}_t, \quad \forall t,$$

$$g_{\min} \le g_t \le g_{\max}, \tag{B.3}$$

$$0 \le r_t, \quad g_t + r_t \le g_{\max},$$

$$|g_t - g_{t-1}| \le R_{\max},$$

$$0 \le p_{\mathrm{ch},t} \le P_{\mathrm{ch,max}}, \tag{B.4}$$

$$0 \le p_{\mathrm{dis},t} \le P_{\mathrm{dis,max}},$$

$$s_t = s_{t-1} + \eta_{\mathrm{ch}} p_{\mathrm{ch},t} \Delta t - \eta_{\mathrm{dis}}^{-1} p_{\mathrm{dis},t} \Delta t, \tag{B.5}$$

$$0 \le s_t \le S_{\max}.$$

and the fixed-proportional reserve protocol

$$\sigma_{w,t} = 0.2 \cdot |\hat{w}_t|, \ t = 1, \ldots, T. \tag{B.6}$$

which is a deterministic comparison rule and not UQ-aware dynamic reserve optimization.

Appendix C. Implementation and environment details

Appendix C.1. Implementation details and computational environment

All experiments are implemented in Python (v3.10) using PyTorch (v2.0+) with CUDA (v11.8) acceleration. The core scientific stack includes NumPy (v1.24+) and Pandas (v2.0+) for data handling, Matplotlib (v3.7+) and Seaborn (v0.12+) for visualization, and LightGBM (v4.0+) with Scikit-learn (v1.3+) for tree-based baselines. These versions are reported to ensure that both deep and non-deep baselines can be reproduced under consistent dependencies. The implementation is openly available at `https://github.com/b1ue13e/Ultra-LSNT` (MIT license), enabling inspection of model components (e.g., sparse routing, skipping gate) and exact reproduction of the reported protocols.

The main training is conducted on a workstation equipped with an NVIDIA GeForce RTX 4090 GPU, an Intel i9-13900K CPU, and 64 GB RAM for reproducible optimization and ablation sweeps. To complement workstation profiling, we report batch-1 inference latency on

commodity CPUs and an ARMv8 (aarch64) single-core platform (Huawei Kunpeng 920). The Kunpeng run is used as a server-class ARM reference for cross-ISA comparison rather than as an edge-device surrogate.

Implementation and training: We implement Ultra-LSNT in native PyTorch without third-party model dependencies. The sparse MoE core uses a custom Top-K routing layer ($K = 2$ in the main model), and the dynamic skip path uses a Jump Gate style conditional pass. To stabilize routing, the released configuration sets the auxiliary load-balancing coefficient to $\lambda = 0.01$ and keeps the router z-loss at 0.01, following standard MoE load-balancing and router-stabilization practice [**?**]; a controlled $\lambda \in \{0.01, 0.05\}$ robustness check is summarized in Table **??**. Unless otherwise stated, we train with $L = 96$, $H = 24$, chronological splits, train-fitted z-score normalization, AdamW $(\beta_1, \beta_2) = (0.9, 0.999)$, weight decay 0.01, cosine annealing with warm-up, initial learning rate 10^{-3}, and batch size 128; typical runs last 10–50 epochs depending on convergence.

Deployment profiling and reproducibility: We profile inference under fixed batch-1 settings for x86/ARM latency screening (Table **??**) and fix seeds plus deterministic settings when possible. Low-level runtime switches, archived manifests, and environment-specific optimization details are documented in the supplementary material rather than repeated in the main manuscript.

Appendix D. Robustness protocol details

Appendix D.1. Perturbation definitions and comparison setup

Additive Gaussian noise: We use additive Gaussian corruption as a controlled sensor-inaccuracy stress test, capturing short-lived measurement noise such as electromagnetic interference and turbulent sensing jitter in SCADA channels. Formally, for a clean input series x, we generate a corrupted series

$$x_{\text{noisy}} = x + \sigma_{\text{eff}} \,\text{std}(x) \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, 1), \tag{D.1}$$

where $\text{std}(x)$ is the standard deviation of the clean signal and σ_{eff} is the effective noise ratio. In our released noise generator, we use an industrial scaling factor $\alpha = 1.5$. To strengthen empirical grounding, we additionally compute a channel-wise proxy noise ratio on Wind (CN) core channels (windspeed, winddirection, temperature, humidity, pressure, power):

$$r_c = \frac{\text{std}(\Delta x_c)}{\sqrt{2}\,\text{std}(x_c)}, \tag{D.2}$$

where $\Delta x_c = x_{t,c} - x_{t-1,c}$. On this dataset, the proxy statistics are median 0.045, P90 0.091, and max 0.121. Detailed calibration artifacts are archived in the supplementary evidence bundle.

Based on this anchor, we treat $\sigma_{\text{eff}} = 0.15$ as an empirical upper-band neighborhood, while $\sigma_{\text{eff}} \in \{0.30, 0.45, 0.60\}$ are explicit stress-envelope settings for robustness certification under abnormal SCADA conditions rather than nominal sensor-error claims. Throughout the manuscript, severity is reported by the *effective* ratio $\sigma_{\text{eff}} = \alpha\sigma_{\text{cfg}}$, so the injected noise has standard deviation $\sigma_{\text{eff}} \cdot \text{std}(x)$. In configuration files and scripts, the user-facing parameter is σ_{cfg}; we therefore report both σ_{cfg} and σ_{eff} when needed. Unless otherwise stated, the Gaussian sweep uses $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ (equivalently $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$).

Structured SCADA faults: Beyond Gaussian noise, real SCADA failures usually show temporal persistence and cross-channel dependence. To reflect that behavior, we synthesize three structured fault families on top of clean streams: flatlining (sensor-freeze windows generated by dwell-time processes such as geometric or semi-Markov sampling), spikes/heavy-tailed outliers (sparse impulsive corruption with amplitudes from Laplace or Student-t distributions), and correlated multi-channel dropouts (common-cause masking events such as DAQ resets that affect multiple variables jointly for contiguous windows). We evaluate these faults on Wind (CN) at severities 0.2 and 0.6; the severe slice is summarized in Table **??**, while full multi-severity rows are moved to supplementary material.

To complement deep-model comparisons, we also run focused non-deep context baselines (LightGBM and Persistence) over configured noise $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ (effective $\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}$ with $\alpha = 1.5$), so severe-slice behavior is inspected with both feature-engineered and naive-history references.

Drift and quantization noise: We also examine two perturbation families that arise naturally in industrial sensing and edge pipelines. Drift noise approximates systematic bias that accumulates over time due to calibration drift, sensor aging, or persistent environmental interference, whereas quantization noise captures rounding and discretization errors introduced by low-precision telemetry, bandwidth-constrained transmission, or edge-side INT8/FP16 processing. To keep perturbation severity comparable, we report both perturbations using the same effective ratio σ_{eff} as in the Gaussian protocol. Unless otherwise stated, we sweep

$$\sigma_{\text{eff}} \in \{0.00, 0.15, 0.30, 0.45, 0.60\}, \tag{D.3}$$

which corresponds to configured $\sigma_{\text{cfg}} \in \{0.00, 0.10, 0.20, 0.30, 0.40\}$ when $\alpha = 1.5$.

Drift (random-walk bias): Given a clean channel $x_{1:L}$, we draw i.i.d. $\varepsilon_t \sim \mathcal{N}(0, 1)$ and define a persistent bias process

$$b_1 = 0. \tag{D.4}$$

$$b_t = b_{t-1} + \sigma_{\text{eff}} \cdot \text{std}(x) \cdot \varepsilon_t, \; t = 2, \ldots, L. \tag{D.5}$$

then corrupt the input as $x_t^{\text{drift}} = x_t + b_t$. The process is applied independently to each feature channel and each test sample. We reset $b_1 = 0$ per sample for paired reproducibility, and use the same random seed across methods at a given σ_{eff}; long contiguous fault persistence is covered separately by the structured-fault protocol.

Quantization (step-size rounding): We implement quantization as deterministic rounding to a channel-wise step size proportional to the clean-signal standard deviation:

$$\Delta = \sigma_{\text{eff}} \cdot \text{std}(x). \tag{D.6}$$

$$x_t^{\text{quant}} = \Delta \cdot \text{round}\left(\frac{x_t}{\Delta}\right). \tag{D.7}$$

where round$(\cdot)$ is round-to-nearest (half away from zero). When $\sigma_{\text{eff}} = 0$, no quantization is applied. This protocol yields a reproducible, severity-controlled discretization without requiring dataset-specific bit-range tuning. We report the corresponding robustness outcomes in Section **??** and interpret their operational implications in the Discussion, with emphasis on reliability under realistic sensing constraints and energy-aware (Green AI) deployment.

Missingness and interpolation artifacts: Communication loss and sensor resets regularly create gaps in SCADA telemetry, after which simple forward-fill or linear interpolation is often applied. To emulate this setting, we sample a binary mask $m_{t,c} \sim \text{Bernoulli}(1 - p)$ over time steps t and channels c and define $\bar{x}_{t,c} = m_{t,c} x_{t,c} + (1 - m_{t,c})\text{NaN}$. We then apply an imputation operator Imp$(\cdot)$ to obtain the corrupted series

$$x^{\text{miss}} = \text{Imp}(\bar{x}; \texttt{ffill/linear}), \tag{D.8}$$

where `ffill` denotes forward filling and `linear` denotes piecewise-linear interpolation along the time axis. In the reported experiments, we use i.i.d. masking (no burst blocks) as a controlled imputation stress test, sweep $p \in \{0.1, 0.3\}$, and repeat each configuration for three random seeds (0,1,2). For a fixed seed, the same missingness mask and imputation configuration are

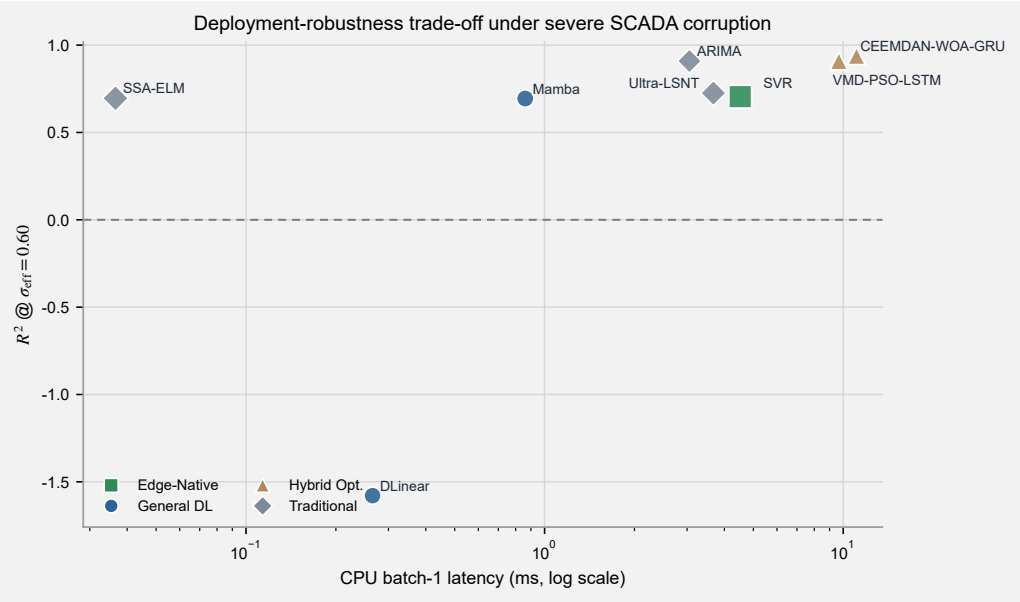


Figure E.7: Deployment-robustness trade-off under severe SCADA corruption ($\sigma_{\text{eff}} = 0.60$). CPU batch-1 latency is shown on the x-axis (log scale), and R^2@0.60 is shown on the y-axis. Point size encodes model size, and colors/markers indicate model family.

applied across models to preserve a paired comparison. Contiguous and correlated missingness is represented by the structured-fault dropout protocol. Results are provided in supplementary material.

Appendix E. Additional deployment and sensitivity figures

Figures **??**–**??** address four supplementary questions: where deployment bottlenecks emerge under severe corruption, how efficiency axes co-vary, how Jump Gate threshold shifts the accuracy-latency trade-off, and how the structural N-K-τ landscape behaves at fixed λ_2. Figure **??** isolates corruption-level deployment bottlenecks, Figure **??** visualizes accuracy-efficiency coupling, Figure **??** tracks threshold-driven accuracy-latency shifts, and Figure **??** summarizes structural Pareto geometry. For Figure **??**, severe-slice R^2 follows the matched-control track and is paired with Panel-A CPU latency from Table **??** as supplementary deployment context.

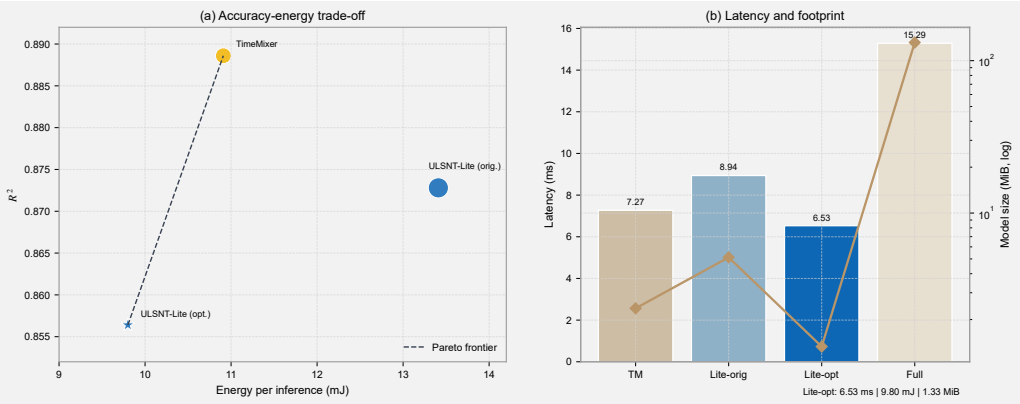


Figure E.8: Deployment-efficiency comparison across Ultra-LSNT variants and principal baselines. Energy, latency, and memory are reported jointly as edge-suitability indicators.

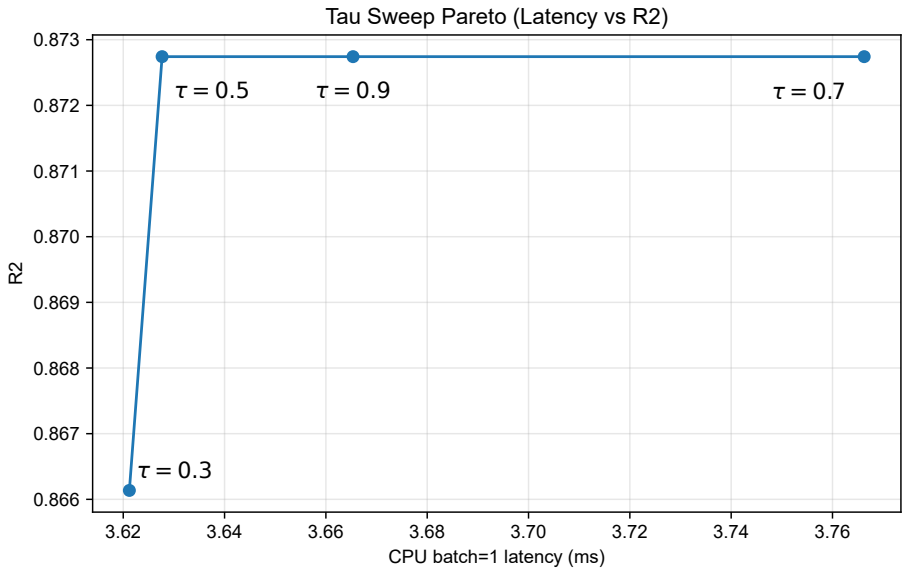


Figure E.9: Jump Gate threshold sweep in supplementary view. The threshold-latency-accuracy trajectory should be read jointly with the archived skip-ratio logs.

Appendix F. Protocol scope and practical limits

Dispatch scope: Main-text dispatch results use one archived mapped-and-clipped forecast-to-grid interface. Supplementary material retains mapping-sensitivity checks, and no interface-independent dispatch claim is made.

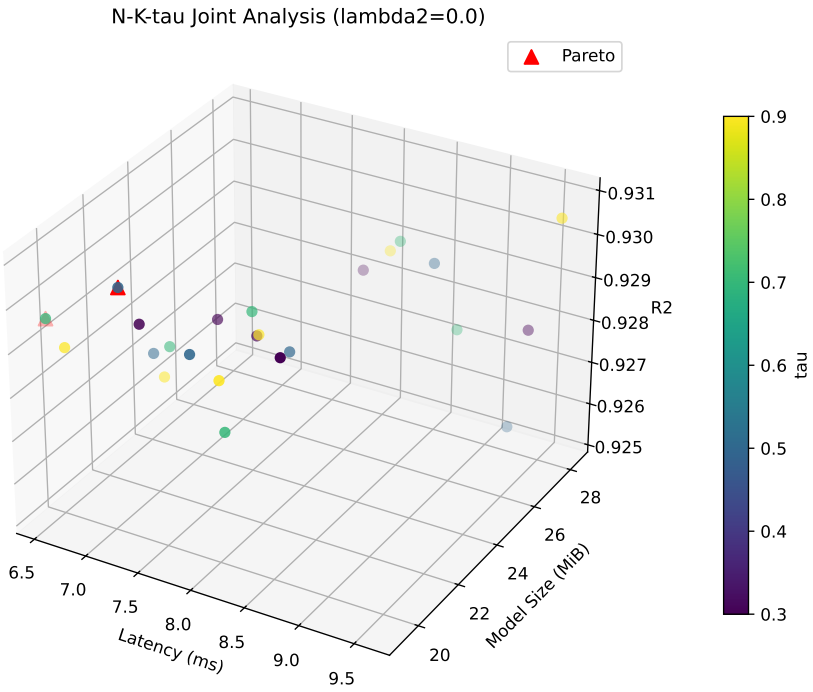


Figure E.10: 3D structural map (N-K-τ) at $\lambda_2 = 0$ with Pareto points highlighted (Wind CN, $\sigma_{\text{eff}} = 0.60$). Main-text decision evidence is reported in Table **??** and Figure **??**.

Hardware scope: Cross-ISA latency profiling is pre-HIL, and board-level validation on target PLC/RTU hardware remains required.

Matched-control scope: This revision adds matched clean/fault-augmented controls for DLinear, TimeMixer, iTransformer, and Ultra-LSNT (three seeds; four faults), but full coverage for long-tail supplementary baselines remains future work.

Transfer scope: Cross-domain transfer results are retained as secondary diagnostic evidence in Table **??**. They are used only to show that no single model dominates all tasks and interfaces.

Appendix G. Supplementary tables: ablations, interpretability, and significance

Appendix G.1. Evidence-file mapping for primary comparative numbers

Primary comparative numbers in this manuscript are generated from consolidated audit tables in the archived supplementary evidence bundle rather than by ad hoc extraction. Detailed file

Table G.7: Supplementary scale-error evidence at clean and $\sigma_{\text{eff}} = 0.60$ for directly logged Wind (CN) rows.

Model	Mode	RMSE (clean)	MAE (clean)	RMSE@0.60	MAE@0.60
DLinear	clean-trained	6065.98	3865.18	22190.19	15183.13
TimeMixer	clean-trained	9081.04	6789.63	12848.91	9674.21
iTransformer	clean-trained	7743.06	6029.31	9933.03	7852.08
Ultra-LSNT	clean-trained	9462.65	7360.92	10992.42	8705.76
DLinear	fault-augmented	6328.26	4181.08	20569.46	14105.63
TimeMixer	fault-augmented	8969.46	7067.89	10129.43	8034.08
iTransformer	fault-augmented	8779.48	6975.59	9593.71	7700.04
Ultra-LSNT	fault-augmented	9925.64	7858.85	10859.03	8675.17
Persistence	clean-trained	7313.94	4760.16	9485.55	6783.51
LightGBM	clean-trained	6880.91	4622.43	17665.50	13740.79
Chronos-Bolt-Small	zero-shot (512 ctx)	6216.19	3818.52	11443.18	8407.16
Moirai-2.0-R-Small	zero-shot (512 ctx)	5834.04	3460.46	11045.80	8176.62

Note: Matched-control rows for DLinear, TimeMixer, iTransformer, and Ultra-LSNT should be read as Panel B companions rather than as Table **??** Panel A context reruns. Chronos and Moirai use zero-shot extended-context inference (512-step input, same 24-step target window) and do not support controller-side deployability claims. Detailed source manifests are provided in the supplementary evidence bundle.

manifests are retained in the public repository and supplement; the main text only relies on those consolidated tables for robustness, safety, and shared-interface dispatch screening, while transfer results are retained as secondary diagnostic evidence.

Appendix G.2. Scale-error supplementary table for robustness audit

Table **??** provides supplementary RMSE/MAE evidence for the Wind (CN) context and matched-control rows used in this revision. The main table already reports RMSE in Panel A; Appendix G retains MAE and protocol-specific row coverage for completeness rather than as a substitute for the primary comparison.

Appendix G.3. Secondary diagnostic transfer table

Table **??** retains the cross-domain transfer audit under the same chronological protocol used in the archived evidence bundle. It is reported as secondary diagnostic evidence rather than as part of the main Wind (CN) claim chain.

Appendix G.4. Full structured SCADA fault robustness table

Complete multi-severity and multi-metric structured-fault results for Wind (CN) are reported in supplementary material and archived in the supplementary evidence bundle.

Table G.8: Secondary diagnostic evidence: cross-domain transferability under a unified chronological protocol.

Model	**Wind (US)** R^2	**Air Quality** R^2	**GEFCom Load** R^2
Ultra-LSNT	0.7878	**0.4861**	0.7554
DLinear	0.8779	0.3433	0.7370
iTransformer	0.8563	0.4590	0.7467
TimeMixer	0.8023	0.3869	0.7201
ARIMA (logged)	0.6971	0.2008	−0.4031
SVR (logged)	0.5796	0.1716	0.8226
SSA-ELM	0.8075	0.3152	0.7662
Mamba (logged)	0.8866	0.3688	−0.0083

Note: Values are taken from the archived supplementary evidence bundle under the same chronological protocol and should be read as boundary-setting diagnostic evidence rather than as an extension of the main matched-control claim.

Appendix G.5. Diagnostic long-tail baseline matrix (diagnostic-only)

For transparency, we retain broad baseline rows, including traditional methods, hybrid methods, and constrained-budget metaheuristics, in the supplementary evidence bundle. These rows are diagnostic only and are excluded from the matched-control main comparisons.

Appendix G.6. Ablation results

All ablations are conducted under a fixed training budget and identical evaluation protocol; we interpret them primarily through relative ranking and directional sensitivity, rather than expecting the absolute R^2 to exactly match the full benchmark runs.

Supplementary ablation tables report kernel-size sensitivity from the archived ablation CSV. On Wind (CN), $k = 3$ and $k = 7$ are close, while $k = 15$ is weaker. This supports using compact receptive fields for turbulence-dominant SCADA dynamics. Router-regularization robustness is reported in Table **??** using the two archived λ checkpoints (0.01 and 0.05).

Appendix G.7. XAI analysis: routing diagnostics for operational audit

Figures **??** and **??** visualize sparse-routing statistics at complementary scales: diurnal/feature correlation and event-window temporal dynamics. This subsection is exploratory governance evidence, not proof of causal physical decomposition. Because routing is regularized by load-balancing objectives, expert activations should be interpreted as correlation-level diagnostics instead of deterministic physical role assignment.

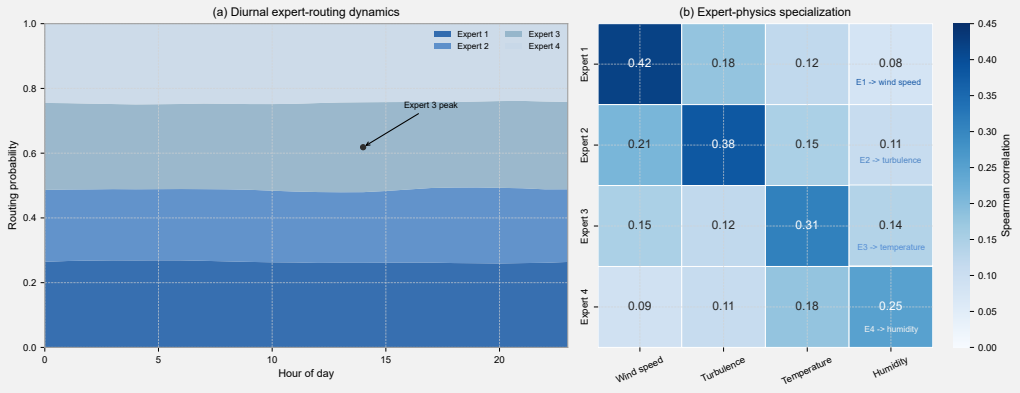


Figure G.11: Interpretability view of sparse routing and expert-feature correlation. The left panel reports diurnal expert-routing composition, and the right panel reports the expert-feature correlation heatmap.

At the diurnal scale, activation shares remain bounded without collapse spikes, which supports auditability of conditional computation. At event windows, routing redistribution is observable and can be logged as a monitoring signal for anomaly triage, but these shifts alone should not be interpreted as direct physical regime identification.

Appendix G.8. Statistical significance

We assess whether performance gaps are statistically significant using the Diebold–Mariano (DM) test on squared prediction errors [**?**]. Let $d_t = \ell_t(\text{Ultra-LSNT}) - \ell_t(\text{Baseline})$ denote the per-time step loss differential computed on the same test split; hence DM< 0 indicates Ultra-LSNT has smaller average loss, while DM> 0 favors the baseline. This significance check is reported to strengthen the auditability of our empirical claims, complementing the descriptive ranking in Table **??**. Figure **??** summarizes the four-domain DM significance matrices used in this revision. Readers should treat the DM view as direction-stability evidence (paired loss differential under fixed splits), not as a replacement for protocol-bound robustness and dispatch screening claims in the main results.

Supplementary DM-test tables indicate statistically significant loss differentials across representative recent backbones. Consistent with Table **??**, the differentials favor TimeMixer and iTransformer under clean measurements (DM> 0), whereas Ultra-LSNT significantly outperforms PatchTST (DM< 0). We also provide scripts and the loss-differential series for the full set of backbones in the supplementary material.

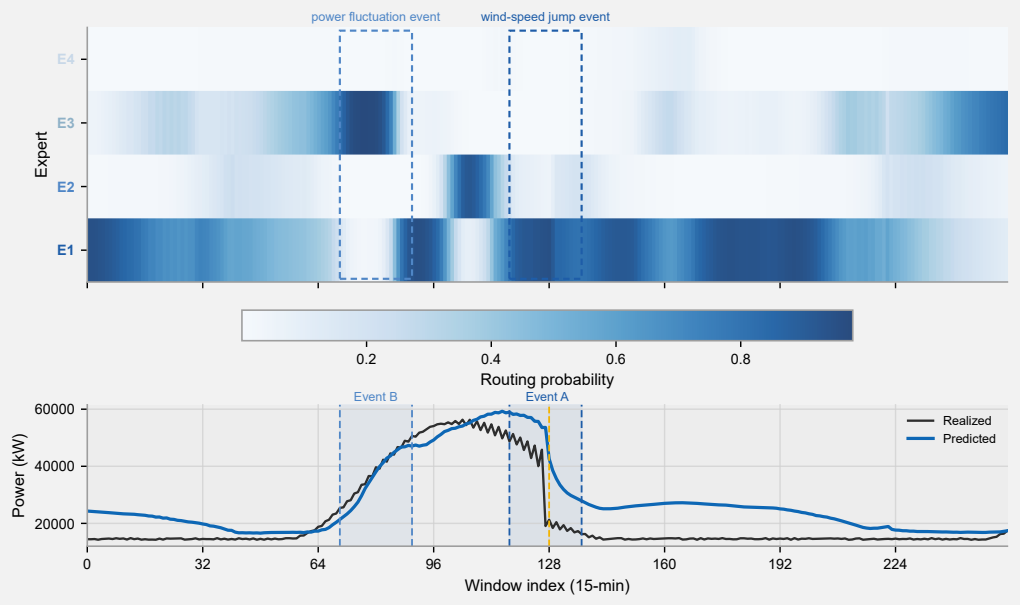


Figure G.12: Time-resolved routing probabilities (4 experts) over a representative 64-hour window with overlaid realized and predicted power. Marked event windows show temporal co-variation between routing redistribution and abrupt trajectory changes; the figure is provided for auditability rather than causal attribution.

Several recent forecasters can achieve higher peak accuracy under clean measurements (Table **??**), but peak clean-data accuracy is not always aligned with stability under realistic sensing imperfections. Accordingly, we report robustness stress tests and downstream dispatch evidence next, where reliability under noise and deployability constraints matter for renewable integration and operational efficiency.

Generative AI and AI-assisted technologies statement

During the preparation of this work, the authors used *ChatGPT* (OpenAI) for language polishing and grammar assistance. The authors performed all scientific interpretation, result reporting, and final manuscript decisions, and they take full responsibility for the content of the publication. No generative AI tools were used to generate or modify experimental data, figures, plots, or graphical abstracts.

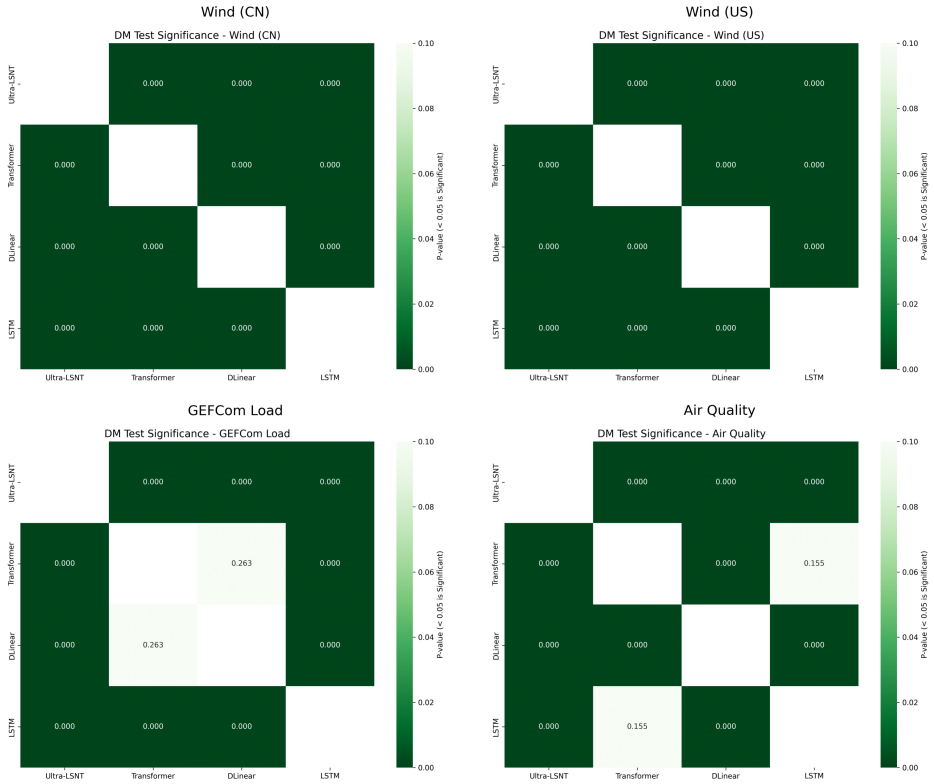


Figure G.13: Direction-aware 2×2 summary of Diebold–Mariano evidence (Ultra-LSNT vs principal baselines) across Wind (CN), Wind (US), GEFCom Load, and Air Quality. Cell entries report direction, p-value, and significance level; complete pairwise DM-statistic magnitude matrices would require dedicated reruns and are not reconstructed here.

Acknowledgments

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.